

Can You Check That? The Checkability Boundary for Local LLM Network Automation

Maleeha Masood

University of Illinois Urbana-Champaign
Urbana, IL, USA

Momina Nofal

Independent Researcher
USA

ABSTRACT

Sending every network-automation input to a third-party frontier LLM exports sensitive artifacts such as production configurations, topologies, and logs, and incurs substantive costs. Querying small language models (SLMs) locally avoids these concerns, but SLM outputs can be too error-prone for direct use. This work introduces *checkability* as a criterion for determining which tasks are suitable for local inference. A task is checkable when it exposes a cheap, deterministic test - an *intrinsic check* - that rejects outputs violating a necessary correctness condition. We instantiate this idea in *Touchstone*, a local-first pipeline that uses seven off-the-shelf SLMs (1–8B parameters) to generate candidates, uses task-specific intrinsic checks to reject responses, and escalates unresolved inputs to a frontier LLM. On conflict detection and intent translation tasks, *Touchstone* reaches 98.6% and 93.8% end-to-end accuracy while escalating only 16% and 17% of inputs, respectively. On TeleQnA, a knowledge-only control that has no task-specific intrinsic checks, *Touchstone* is unable to match the accuracy of the frontier baseline. Our results support a simple deployment rule: keep inference local when task semantics support precise, low-cost checks; escalate the rest.

1 INTRODUCTION

Network operators are unlikely to rely entirely on a model they cannot expose their data to, cannot inspect, or simply, cannot afford to run. This is the uncomfortable gap in today’s LLM-driven network automation. Frontier-scale models have made ambitious tasks like translating operator intent, triaging incidents, detecting anomalies, generating configurations, and synthesizing control code more feasible [8, 12, 13, 19, 21, 23, 27, 29]. Yet, LLM inference often requires exporting sensitive operational artifacts to third-parties, while incurring substantial inference costs. On-premise inference can avoid this sharing of data, but running an LLM requires significant GPU infrastructure and operational expertise. This leaves network automation in a paradoxical state where the strongest models are often least usable where their help would matter most.

Small language models (SLMs) i.e. language models with ≤ 8 B parameters, appear to close this gap. An SLM can run

inside an operator’s environment, keep sensitive data within the network boundary, and avoid the cost and governance burden of remote frontier inference [4]. However, SLMs are known to have weaker reasoning, are more prone to hallucinations and have smaller context windows than LLMs [21, 26, 27]. Thus, treating an SLM as merely a smaller frontier LLM is the wrong abstraction. SLMs can only be safely deployed for network automation when there is a principled rule for admitting local outputs and escalating the remainder to a frontier LLM.

We introduce *intrinsic checks* to decide if a response from an SLM is safe to use. An intrinsic check is a task-specific, deterministic predicate over the task input, a candidate output, and optional network state that tests a necessary condition for correctness.

Many network automation tasks expose such intrinsic checks. A reachability specification should not both permit and deny the same location–prefix pair. An intent translation should not introduce locations or prefixes absent from the request. A log template should regenerate the concrete log line it summarizes. Routing code can be executed against tests on concrete topologies. Intrinsic checks are simple, inexpensive checks and do not compare against a reference answer, or invoke a verifier or LLM.

We instantiate *intrinsic checks* in *Touchstone*¹, a local-first pipeline that uses seven heterogeneous, off-the-shelf SLMs with 1–8B parameters. For each input, the SLMs generate candidates, and task-specific intrinsic checks reject candidates that violate necessary conditions. *Touchstone* then selects from remaining candidates or escalates input when no candidate passes the intrinsic checks. In *Touchstone*, the SLMs provide diversity, intrinsic checks determine eligibility for accepting an SLM’s response, and frontier escalation handles the unresolved residue. Instead of completely eliminating third-party inference, *Touchstone* just makes it selective.

We evaluate *Touchstone* on 4 different structured networking tasks, namely conflict detection, intent translation, log parsing, and generating routing code, and a knowledge based dataset on telecommunications called TeleQnA. We report

¹A touchstone is a test or criterion for determining the quality or genuineness of a thing. Historically, it is a stone used to test the purity of precious metals.

end-to-end accuracy and frontier LLM escalation rate, and compare against a baseline that sends every input to the frontier LLM. Touchstone reaches 98.6% end-to-end accuracy on conflict detection with 16% escalation and zero false accepts, and 93.8% accuracy on intent translation with 17% escalation. On TeleQnA, a knowledge-only control for which there are no task-specific intrinsic checks, Touchstone remains below the frontier baseline. This negative result is instructive: model diversity and selective escalation alone cannot match all-frontier performance unless the task exposes intrinsic checks. Together, these results support using intrinsic checks as a practical deployment boundary for local-first LLM network automation.

This paper makes three contributions:

- We identify *checkability* as the boundary for local deployment: tasks whose outputs can be cheaply and deterministically checked should remain local, while tasks without such checks should be escalated to a frontier LLM.
- We define *intrinsic checks* as deterministic tests that reject a candidate when it makes a claim the task input or network state refutes.
- We evaluate Touchstone on four structured networking tasks and a knowledge-only control, quantifying the end-to-end accuracy–escalation tradeoff.

2 INTRINSIC CHECKS

Definition: Intrinsic Check

Let $(P(x,y))$ mean “ y is correct for input x .” An *intrinsic check* is a deterministic predicate $Q(x, y, s)$, with optional network state s , such that correctness implies the check:

$$P(x, y) \implies Q(x, y, s).$$

That is, Q is a *necessary* condition for correctness, but not a sufficient one.

An intrinsic check uses no ground truth, LLM, learned verifier, or remote API. Since Q is only necessary, it is one-sided:

- A **failed** check ($\neg Q$) entails $\neg P$: the candidate is refuted, because it violates a condition every correct answer must satisfy.
- A **passed** check (Q) only means the candidate survived refutation. Its reliability depends on the *false-accept rate* (FAR)—the fraction of accepted candidates that are wrong—which must be measured.

Many network automation tasks produce artifacts with constraints that any correct answer must satisfy. These constraints for correctness can be tested for with cheap deterministic tests that we call *intrinsic checks*.

We use existing networking datasets to generate task-specific intrinsic checks.

Intent translation. In NetConfEval formal-specification translation [27], the task is to compile natural-language requirements into a JSON specification containing reachability, waypoint, and load-balancing rules. The simplest syntactic check is that the output must be a dictionary with the three expected keys. The intrinsic checks however go beyond the shape of the object and ask whether its contents are supported by the request. For example, the input

```
Traffic originating from barcelona can reach the subnet
100.0.9.0/24.
```

should produce a specification such as:

```
{"loadbalancing": {},
 "reachability": {"barcelona": ["100.0.9.0/24"]},
 "waypoint": {}}
```

We develop two intrinsic checks for this task. First, the output should be *grounded*: every prefix and location in the generated specification should appear in the input request. Second, the output should provide *coverage*: the number of generated rule atoms should roughly match the number of requirement clauses in the request. These checks reject common SLM failures such as hallucinating unseen subnets, or silently dropping large portions of the request. However, they do not prove that a grounded and complete-looking specification correctly captures the operator’s intent.

Conflict detection. In NetConfEval translation conflict detection [27], the task is to decide whether a set of natural-language requirements contains a contradiction. For e.g.:

```
The subnet 100.0.22.0/24 is reachable from amsterdam.
geneva cannot reach 100.0.4.0/24.
100.0.4.0/24 is accessible from geneva.
```

The correct output is: CONFLICT

The request is conflicting because the same $\langle \text{location}, \text{prefix} \rangle$ pair is asserted both reachable and unreachable. The intrinsic check for this task extracts reachability predicates and tests *consistency*. Specifically, it parses each requirement sentence into $\langle \text{location}, \text{prefix} \rangle$ pairs, classifies each as reachable or unreachable using the text in the input, and flags a conflict whenever the same pair is asserted both ways, deterministically catching contradictions. If the input contains a conflict, any candidate that says “no conflict” is rejected. If no contradiction is extracted, the check does not prove the request is conflict-free.

Log parsing. In Loghub’s OpenSSH, HDFS and Proxifier datasets [30], the task is to convert a raw log line into its event template, replacing every variable value (IPs, IDs, hostnames, ports) with a wildcard $\langle * \rangle$. For example:

```
Failed password for invalid user admin from 5.188.10.180 port
60682 ssh2
```

should produce a template such as:

```
Failed password for invalid user <*> from <*> port <*> ssh2
```

The intrinsic check we develop is invertibility: the generated template must be able to regenerate the original log line by replacing wildcards with concrete substrings.

This check is a valid Q, but with a larger false-accept rate, since many over-general templates can still regenerate the line.

Routing code generation. In NetConfEval’s Routing Code Generation task [27], the model must write a Python function that returns the routing paths between hosts. For example, for the shortest-path policy and a topology in which hosts h1 and h2 share a LAN A, the function must satisfy:

```
route({'h1': {0: 'A'}, 'h2': {0: 'A'}})
== {'h1': {'h2': ['h1', 'h2']},
    'h2': {'h1': ['h2', 'h1']}}
```

and the model should produce a function such as:

```
def route(topology, requirements=None):
    lan = {}
    for d, ports in topology.items():
        for l in ports.values():
            lan.setdefault(l, []).append(d)
    adj = {d: {n for l in p.values()
               for n in lan[l] if n != d}
           for d, p in topology.items()}
    # BFS shortest paths between host pairs
    ...
    return paths
```

The intrinsic check here is *execution*: the generated function is run against the task’s test cases. Unlike the checks above, execution-based checking is already standard practice for code. We include it as the strongest example of intrinsic checking

Intrinsic checks are cheap to execute because they compare a candidate’s claims against facts or constraints already available in the input or network state using ordinary deterministic operations. Each check defines an admission boundary: candidates for which no error witness is found may be accepted locally, while the remaining candidates are escalated. However, passing a check does not prove that a candidate is correct. An incorrect candidate may pass when its error leaves no detectable witness. We therefore characterize each check by its false-accept rate, measured separately for each task in §4.1.

Uncheckable Tasks. A useful intrinsic check works because passing it rules out a class of wrong answers. A passing candidate has survived a necessary condition, so it is less likely to be wrong. This depends on the task exposing a property that correct and incorrect answers do not share. We find that knowledge-based tasks such as TeleQnA [18], which tests telecommunications knowledge, expose no such property. Any answer choice is a well-formed, valid option, but it can still be wrong, because its correctness depends

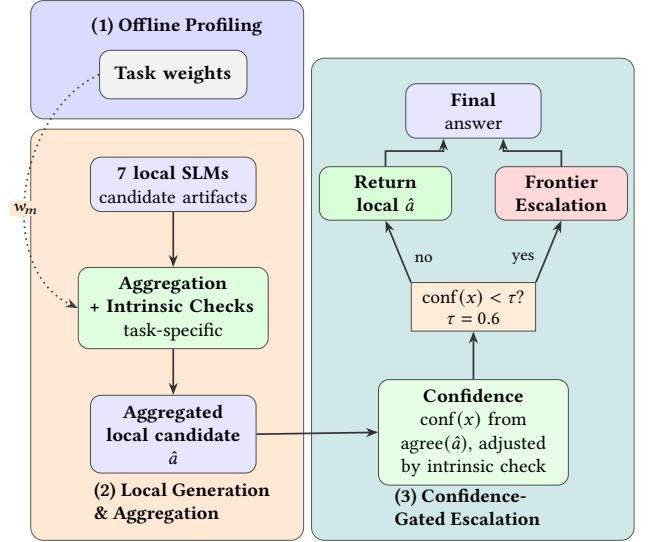


Figure 1: Touchstone’s local-first workflow

entirely on external knowledge rather than task-intrinsic structure.

Who Writes the Checks? Intrinsic checks are written once per task by the system designer, rather than per query by the operator. In our prototypes, these checks are small: the grounding-and-coverage check for intent translation requires 49 lines of code, the check for conflict detection requires 43, and the invertibility check for log parsing requires 28. The designer does not need to solve the task or specify every valid output. Instead, the designer identifies necessary conditions that any valid output must satisfy, such as using only prefixes present in the request or preserving required way-points. Each condition must be a sound refuter: violating it must provide valid grounds for rejecting the candidate. The resulting check is then evaluated on a development set using two quantities: its false-accept rate, which captures incorrect candidates that pass, and its unnecessary-escalation rate, which captures correct candidates that are rejected. These measurements make the check’s residual risk and loss of local coverage explicit before deployment.

3 THE TOUCHSTONE WORKFLOW

Touchstone is an admission-control architecture for local SLM inference with intrinsic checks. We say that a candidate is *admitted* when the system accepts it without escalating the input to a frontier LLM. Touchstone works by separating generation from admission: a family of SLMs proposes candidate answers, and intrinsic checks reject candidates that violate necessary conditions. A confidence score then decides if the local answer is admissible or if the query needs to be escalated to a frontier LLM to resolve.

Touchstone runs seven off-the-shelf SLMs—Llama-1B, Qwen-1.5B, SmolLM-1.7B, Qwen-3B, Llama-3B, Qwen-7B, and Llama-8B—in three phases: (1) offline profiling, (2) local generation and aggregation, and (3) confidence-gated escalation.

Offline profiling. Touchstone profiles each SLM once on a small held-out development split. This task-specific accuracy is then used to determine the model’s competence on the task and consequently, its voting weight w_m . Our development sets contain 23.1–25.0% of the available data i.e. 100-150 examples for each task.

For each model m , Touchstone computes

$$w_m = \max(0.02, \text{acc}_m - \text{chance})^{1.5},$$

where acc_m is the model’s accuracy on the corresponding development split and chance is the random baseline (0.5 for conflict detection, 0 for the open-ended generation tasks). Subtracting chance prevents near-random models from dominating the aggregate, and the exponent emphasizes models that are reliably above chance.

Local generation and aggregation. For each input, Touchstone queries all seven SLMs and aggregates their outputs using a task-specific procedure. The aggregation method depends on the structure of the output: conflict detection and log parsing produce a single answer, intent translation produces a set of rules, and routing-code generation may admit multiple correct programs.

For conflict detection and log parsing, Touchstone uses a *skill-weighted vote*. Each model m produces a prediction with weight w_m . For every distinct prediction, Touchstone sums the weights of the models that produced it and selects the prediction with the largest total weight. The intrinsic check constrains this selection. For conflict detection, a consistency check overrides the vote when it establishes a contradiction. For log parsing, an invertibility check excludes templates that cannot regenerate the original log line.

Routing-code generation requires no voting because several implementations may be correct. Instead, Touchstone executes every generated function against the test suite and retains each candidate that passes the execution check.

Intent translation requires finer-grained aggregation because each output is a set of rules, and a model may generate some rules correctly while missing or misgenerating others. Touchstone therefore decomposes every generated specification into *atoms*, with one atom per rule. For example, the rule that Paris must reach prefix $100.0.1.0/24$ becomes ("reach", "paris", " $100.0.1.0/24$ "). Touchstone pools atoms across all seven models, merges identical atoms, and retains an atom when the models that generated it account for at least half of the ensemble’s total weight. Thus, an atom may be retained through agreement among several models or through the support of one highly weighted model.

This rule-level aggregation can construct a correct specification from complementary outputs even when no individual model produces the entire specification correctly. However, atom-wise recombination can also introduce an incoherent combination of individually plausible rules. To guard against this failure, Touchstone compares the assembled specification with the individual specification that scored highest on the intrinsic checks and retains whichever of the two that receives the higher grounding-and-coverage score.

At the end of this phase, Touchstone produces one aggregated candidate for conflict detection, log parsing, and intent translation. For routing-code generation, it retains all candidates that pass the execution check.

Confidence-gated escalation. After aggregation, Touchstone assigns the selected local candidate a confidence score $\text{conf}(x) \in [0, 1]$. This score combines two signals: the weighted support of the SLM family and the evidence supplied by the task’s intrinsic check. Touchstone returns the local candidate when its confidence meets a threshold τ and otherwise escalates to a frontier LLM:

$$\text{escalate}(x) \iff \text{conf}(x) < \tau, \quad \tau = 0.6.$$

Thus, escalation is a single per-input decision made only after aggregation.

The base confidence is the *family agreement*: the fraction of the ensemble’s total skill weight supporting the selected candidate. For tasks with a single-valued output,

$$\text{agree}(\hat{a}) = \frac{\sum_{m: \text{pred}_m = \hat{a}} w_m}{\sum_m w_m},$$

where $\hat{a}$ is the aggregated prediction. For intent translation, whose output is assembled from multiple rule atoms, Touchstone instead uses the mean weighted support of the atoms in the assembled specification.

The intrinsic check then modifies this base confidence. The exact rule is task-specific because the checks provide different strengths of evidence.

For log parsing, Touchstone first excludes templates that fail the invertibility check and uses the family agreement of the selected passing template as its confidence. For intent translation, the grounding-and-coverage check identifies concrete defects such as entities not grounded in the request or missing required rules. Such defects discount the agreement score and can push an otherwise well-supported specification below the escalation threshold.

For conflict detection, the check is one-sided: it can prove that two policies conflict, but it cannot prove that they are consistent. When the check establishes a contradiction, Touchstone selects CONFLICT and sets $\text{conf}(x) = 1$. Otherwise, confidence remains the weighted agreement behind the selected label. For routing code, execution provides a stronger acceptance signal than model agreement. Touchstone therefore

Dataset	<i>n</i>	Best SLM	GPT-5.5	Touchstone	Egress
<i>Structured Tasks</i>					
Conflict det.	500	0.840	1.000	0.986	16%
Intent transl.	500	0.740	0.992	0.938	17%
Log parse (SSH)	300	0.743	0.997	0.997	37%
Log parse (HDFS)	300	0.863	0.987	0.917	7%
Log parse (Prox.)	300	0.650	0.917	0.877	42%
Routing code	8	0.125	0.500	0.500	75%
<i>Knowledge Tasks</i>					
TeleQnA	500	0.780	0.842	0.782	16%

Table 1: Touchstone accuracy and frontier egress across all tasks, compared with GPT-5.5 and Best Single SLM accuracy.

Check	Task	FAR
Consistency	Conflict detection	0.0%
Grounding + coverage	Intent translation	5.6%
Invertibility	Log (OpenSSH)	27.7%
Invertibility	Log (HDFS)	14.4%
Invertibility	Log (Proxifier)	30.0%

Table 2: False-accept rate (FAR) of each intrinsic check on the evaluation split.

sets confidence to 1 when at least one generated function passes the test suite and to 0 when none does.

When Touchstone escalates an input, it includes the check’s diagnostic evidence—for example, an unreproduced token, an ungrounded entity, or a failed execution test—in the frontier-model prompt as a repair hint. Touchstone returns the frontier model’s reponse as the final output.

4 EVALUATION

We evaluate Touchstone based on its accuracy, egress (the percentage of input getting escalated to the frontier LLM, GPT-5.5), and the false-accept rates of the intrinsic checks of each task.

We use four existing structured networking task datasets: conflict detection, intent translation, log parsing, and routing-code generation. These cover logical, structural, invertibility-based, and execution-based intrinsic checks (§2). We also include TeleQnA [18] as a knowledge-only control which tests the boundary case where Touchstone has no intrinsic check to apply.

Table 1 reports accuracy and egress against an all-frontier baseline. Table 2 reports false-accept rate (FAR) of each task’s checks: among items the check accepted locally, what fraction were actually wrong. This isolates check reliability.

4.1 Accuracy, Egress and False-accept Rates

Touchstone performs best on the two tasks with the lowest false-accept rates. On conflict detection, it reaches 98.6% accuracy, compared with 100% for GPT-5.5, while escalating only 16% of inputs. We observe no false accepts: when the input contains an explicit contradiction, the check rejects any candidate that incorrectly reports no conflict. On intent translation, Touchstone reaches 93.8% accuracy while escalating 17% of requests, compared with 99.2% accuracy when every request is sent to GPT-5.5. Its 5.6% false-accept rate reflects the limits of the grounding and coverage checks: they catch hallucinated entities and omitted clauses, but do not prove that the generated policy is semantically correct.

Log parsing shows the limits of a weak check. Invertibility is necessary, but not very selective: an over-general template can still regenerate the original log line and therefore pass. As a result, the false-accept rate is high—27.7% on OpenSSH and 30.0% on Proxifier i.e. nearly one-third of templates that pass the check are still incorrect. Touchstone compensates by escalating 37% and 42% of inputs, respectively. Its accuracy therefore comes largely from frontier fallback rather than from the check itself, which is too permissive to serve as a reliable admission signal.

HDFS illustrates a different failure mode. It escalates only 7% of inputs—the lowest rate among the structured tasks—but reaches 91.7% accuracy, compared with 98.7% for the frontier model. Its 14.4% false-accept rate explains the gap: low egress is useful only when locally admitted outputs are reliable. Keeping incorrect answers local is not a success, even if it reduces frontier use.

Routing code and TeleQnA expose opposite limits of local admission. Routing code has the strongest check: execution can reliably reject incorrect programs. However, the SLMs rarely generate a function that passes—only 1 of 8 inputs for the best single model and 2 of 8 across all models—so Touchstone escalates 75% of requests and reaches only the frontier model’s 50% accuracy. A strong check is therefore useful only when the local models can generate valid candidates. TeleQnA has the opposite problem. Because an incorrect answer is just as well formed as the correct one, the task exposes no intrinsic witness of error. Even the frontier model solves only 84.2% of the questions, showing that the remaining errors depend on missing knowledge rather than detectable structural violations. Without a task-specific check, Touchstone reaches 78.2% accuracy while escalating 16% of requests.

4.2 Improvement over the best single SLM

We compare the best single SLM with the full Touchstone pipeline. On conflict detection, Touchstone improves accuracy from 84.0% to 98.6%, while on intent translation it improves accuracy from 74.0% to 93.8%. The gains vary across

the three log-parsing datasets: accuracy increases from 74.3% to 99.7% on OpenSSH, from 86.3% to 91.7% on HDFS, and from 65.0% to 87.7% on Proxifier. On TeleQnA, where no intrinsic check is available, accuracy changes only marginally, from 78.0% to 78.2%. These results show that Touchstone provides the largest improvements on checkable tasks.

5 INTRINSIC CHECK AS A BOUNDARY

Our experience with Touchstone suggests that the central question for local SLM network automation is not “*which model is strong enough?*” but “*which tasks are checkable enough?*” This reframes local deployment as a property of the task, not just the model. A small model can be useful when the task exposes deterministic structure that makes it easy to reject bad outputs. The same model is unsafe when the task requires external knowledge or judgment that cannot be checked from the input and local state.

This checkability boundary is a spectrum dependent on strength of the intrinsic checks. At one end are tasks with strong deterministic tests, such as conflict detection with explicit logical contradictions. In the middle are tasks such as intent translation, where grounding and coverage reject common failures but cannot prove policy equivalence. At the weak end are tasks such as log parsing, where invertibility is useful but permissive. Outside the boundary are knowledge-only tasks, where the prompt and candidate answer provide no deterministic way to identify correctness.

6 RELATED WORK

LLM-driven network automation. Recent work has explored LLMs for network automation and networking tasks [8, 12, 13, 19, 21, 23, 27, 29]. These systems show that language models can produce useful network artifacts, but they also expose a deployment tension: the strongest results often rely on frontier models, tool-rich workflows, or model-specific adaptations. Our work asks a different question: when can inference remain local without exposing sensitive operator data? Touchstone targets that boundary. It uses SLMs as candidate answer generators and uses intrinsic checks to decide which candidates are admissible.

Constraints, feedback, and ambiguity in LLM-based networking. Verified Prompt Programming combines GPT-4 with syntax and semantic verifiers, plus human input, to synthesize router configurations [21]. Clarify shows that configuration synthesis also requires eliciting ambiguous operator intent, especially when route maps and ACLs overlap in header space and priority cannot be inferred from the prompt alone [20]. LeJIT interleaves an SMT solver with generation so model outputs obey domain-specific logic during inference [11].

Touchstone shares the view that model outputs need external structure, but uses that structure for a different purpose. Intrinsic checks are cheap deterministic admission tests applied after local generation. A failed check rejects a candidate; a passed check only says the candidate has not violated the tested necessary condition.

Network verification, synthesis, and checking. Network verification systems check properties such as reachability, isolation, equivalence, policy compliance, and change safety [1, 2, 5, 7, 10, 15–17, 25]. Network-synthesis systems similarly compile or complete configurations from structured objectives [3, 9, 24]. Touchstone does not compete with this line as a verification system. It is weaker by design: its checks are per-query necessary-condition tests used to gate local LLM outputs before they reach downstream automation.

Model cascades and ensembles. Model cascades systems reduce cost by routing queries among models, aggregating generations, or falling back when confidence is low [6, 14, 22, 28]. We share the generate-and-check philosophy, but use deterministic network checks for a different purpose: to decide when local inference is admissible. A candidate is accepted only when it satisfies external network constraints, not because a model is confident or a majority agrees. Cases that local checks cannot resolve are escalated. Thus, checkability, not ensembling, is the operative boundary.

7 CONCLUSION

Local-first network automation cannot rely on SLM accuracy or model agreement alone. We argue that its viability depends on whether the task exposes cheap, deterministic checks that can reject erroneous outputs. Touchstone instantiates this idea by using SLMs to generate local candidates, admitting only those that survive task-specific intrinsic checks, and escalating the remainder to a frontier LLM. When the checks have low false-accept rates and the local models can generate valid candidates, Touchstone keeps most requests on premises while approaching frontier-model accuracy. Our results support a simple deployment rule: keep inference local when task semantics support precise, low-cost checks; escalate the rest.

REFERENCES

- [1] Carolyn Jane Anderson, Nate Foster, Arjun Guha, Jean-Baptiste Jeannin, Dexter Kozen, Cole Schlesinger, and David Walker. 2014. NetKAT: semantic foundations for networks. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Diego, California, USA) (POPL ’14). Association for Computing Machinery, New York, NY, USA, 113–126. <https://doi.org/10.1145/2535838.2535862>
- [2] Ryan Beckett, Aarti Gupta, Ratul Mahajan, and David Walker. 2017. A General Approach to Network Configuration Verification. In *Proceedings of the Conference of the ACM Special Interest Group on*

- Data Communication* (Los Angeles, CA, USA) (*SIGCOMM '17*). Association for Computing Machinery, New York, NY, USA, 155–168. <https://doi.org/10.1145/3098822.3098834>
- [3] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, and David Walker. 2016. Don't Mind the Gap: Bridging Network-wide Objectives and Device-level Configurations. In *Proceedings of the 2016 ACM SIGCOMM Conference* (Florianopolis, Brazil) (*SIGCOMM '16*). Association for Computing Machinery, New York, NY, USA, 328–341. <https://doi.org/10.1145/2934872.2934909>
- [4] Rina Diane Caballar and Cole Stryker. 2026. What are small language models? IBM Think. <https://www.ibm.com/think/topics/small-language-models> Accessed: 2026-07-05.
- [5] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. CodeT: Code Generation with Generated Tests. <https://doi.org/10.48550/ARXIV.2207.10397>
- [6] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176 [cs.LG] <https://arxiv.org/abs/2305.05176>
- [7] Marcos Cramer and Lucian McIntyre. 2025. Verifying LLM-Generated Code in the Context of Software Verification with Ada/SPARK. arXiv:2502.07728 [cs.SE] <https://arxiv.org/abs/2502.07728>
- [8] Rohit Dwivedula, Divyanshu Saxena, Aditya Akella, Swarat Chaudhuri, and Daehyeok Kim. 2025. Man-Made Heuristics Are Dead. Long Live Code Generators!. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks* (UMD Campus, College Park, MD, USA) (*HotNets '25*). Association for Computing Machinery, New York, NY, USA, 51–60. <https://doi.org/10.1145/3772356.3772413>
- [9] Ahmed El-Hassany, Petar Tsankov, Laurent Vanbever, and Martin Vechev. 2018. NetComplete: Practical Network-Wide Configuration Synthesis with Autocompletion. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, Renton, WA, 579–594. <https://www.usenix.org/conference/nsdi18/presentation/el-hassany>
- [10] Ari Fogel, Stanley Fung, Luis Pedrosa, Meg Walraed-Sullivan, Ramesh Govindan, Ratul Mahajan, and Todd Millstein. 2015. A general approach to network configuration analysis. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. 469–483.
- [11] Hongyu He and Maria Apostolaki. 2025. Just-in-Time Logic Enforcement: A new paradigm of combining statistical and symbolic reasoning for network management. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks* (UMD Campus, College Park, MD, USA) (*HotNets '25*). Association for Computing Machinery, New York, NY, USA, 184–192. <https://doi.org/10.1145/3772356.3772406>
- [12] Zhiyuan He, Aashish Gottipati, Lili Qiu, Xufang Luo, Kenuo Xu, Yuqing Yang, and Francis Y. Yan. 2024. Designing Network Algorithms via Large Language Models. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks* (Irvine, CA, USA) (*HotNets '24*). Association for Computing Machinery, New York, NY, USA, 205–212. <https://doi.org/10.1145/3696348.3696868>
- [13] Md. Kamrul Hossain and Walid Aljoby. 2025. NetIntent: Leveraging Large Language Models for End-to-End Intent-Based SDN Automation. *IEEE Open Journal of the Communications Society* 6 (2025), 10512–10541. <https://doi.org/10.1109/OJCOMS.2025.3642642>
- [14] Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. 2023. LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 14165–14178. <https://doi.org/10.18653/v1/2023.acl-long.792>
- [15] Peyman Kazemian, Michael Chang, Hongyi Zeng, George Varghese, Nick McKeown, and Scott Whyte. 2013. Real Time Network Policy Checking Using Header Space Analysis. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*. USENIX Association, Lombard, IL, 99–111. <https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/kazemian>
- [16] Alexander Krentsel, Oliver Ye, Anthony Tafoya, Xuqian Ma, Sylvia Ratnasamy, and Anees Shaikh. 2025. Towards Accessible Model-Free Verification. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks* (UMD Campus, College Park, MD, USA) (*HotNets '25*). Association for Computing Machinery, New York, NY, USA, 210–217. <https://doi.org/10.1145/3772356.3772380>
- [17] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (Dec. 2022), 1092–1097. <https://doi.org/10.1126/science.abq1158>
- [18] Ali Maatouk, Fadel Ayed, Nicola Piovesan, Antonio De Domenico, Merouane Debbah, and Zhi-Quan Luo. 2026. TeleQnA: A Benchmark Dataset to Assess Large Language Models Telecommunications Knowledge. *IEEE Network* 40, 2 (2026), 253–260. <https://doi.org/10.1109/MNET.2025.3576035>
- [19] Sathiy Kumar Mani, Yajie Zhou, Kevin Hsieh, Santiago Segarra, Trevor Eberl, Eliran Azulai, Ido Frizler, Ranveer Chandra, and Srikanth Kandula. 2023. Enhancing Network Management Using Code Generated by Large Language Models. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks* (Cambridge, MA, USA) (*HotNets '23*). Association for Computing Machinery, New York, NY, USA, 196–204. <https://doi.org/10.1145/3626111.3628183>
- [20] Rajdeep Mondal, Nikolaj Björner, Todd Millstein, Alan Tang, and George Varghese. 2025. Tackling Ambiguity in User Intent for LLM-based Network Configuration Synthesis. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks* (UMD Campus, College Park, MD, USA) (*HotNets '25*). Association for Computing Machinery, New York, NY, USA, 176–183. <https://doi.org/10.1145/3772356.3772402>
- [21] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, and George Varghese. 2023. What do LLMs need to Synthesize Correct Router Configurations?. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks* (Cambridge, MA, USA) (*HotNets '23*). Association for Computing Machinery, New York, NY, USA, 189–195. <https://doi.org/10.1145/3626111.3628194>
- [22] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. 2024. RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665 [cs.LG] <https://arxiv.org/abs/2406.18665>
- [23] Alagappan Ramanathan, Eunju Kang, Dongsu Han, and Sangeetha Abdu Jyothi. 2025. Towards an Agentic Workflow for Internet Measurement Research. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks* (UMD Campus, College Park, MD, USA) (*HotNets '25*). Association for Computing Machinery, New York, NY, USA, 61–68. <https://doi.org/10.1145/3772356.3772409>
- [24] Sivaramakrishnan Ramanathan, Ying Zhang, Mohab Gawish, Yogesh Mundada, Zhaodong Wang, Sangki Yun, Eric Lippert, Walid Taha, Minlan Yu, and Jelena Mirkovic. 2023. Practical Intent-driven Routing Configuration Synthesis. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 629–644. <https://www.usenix.org/conference/nsdi23/presentation/ramanathan>

- [25] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PI-CARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 9895–9901. <https://doi.org/10.18653/v1/2021.emnlp-main.779>
- [26] Chongren Sun, Yuran Li, Di Wu, and Benoit Boulet. 2025. OnionEval: An Unified Evaluation of Fact-conflicting Hallucination for Small-Large Language Models. arXiv:2501.12975 [cs.CL] <https://arxiv.org/abs/2501.12975>
- [27] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, and Marco Chiesa. 2024. NetConfEval: Can LLMs Facilitate Network Configuration? *Proc. ACM Netw.* 2, CoNEXT2, Article 7 (June 2024), 25 pages. <https://doi.org/10.1145/3656296>
- [28] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171 [cs.CL] <https://arxiv.org/abs/2203.11171>
- [29] Duo Wu, Xianda Wang, Yaqi Qiao, Zhi Wang, Junchen Jiang, Shuguang Cui, and Fangxin Wang. 2024. NetLLM: Adapting Large Language Models for Networking. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) (*ACM SIGCOMM '24*). Association for Computing Machinery, New York, NY, USA, 661–678. <https://doi.org/10.1145/3651890.3672268>
- [30] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. 2023. Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. 355–366. <https://doi.org/10.1109/ISSRE59848.2023.00071>